

A Comparative Study of Multilayer Perceptron and Support Vector Machine on Water Quality

Introduction:

Water is an important natural resource which the planet depends on as it contributes to 71% of the earth's area [1] and is a key part to living organisms' life. As well as that, it is an essential resource in industries such as agriculture, food production and industrial resources. The many uses of water have different quality requirements; for example, water used for drinking water must be fresh and unpolluted.

This paper will look to critically evaluate two neural computing methods models in the aim to identify whether a particular water source is safe to drink or not. The two methods that shall be looked at in this paper are Feedforward Multilayer perceptron and Support Vector Machines, where we will look to find the best performing model for each.

Multiplayer perceptron (MLP):

A Multi-layer perceptron (MLP) is a type of artificial neural network that is composed of multiple layers, which includes an input layer, one or more hidden layers and an output layer. The connections between the layers are assigned weights which are adjusted during the training process. The weight of a connection specifies its importance. A popular method of training an MLP is backpropagation which adjusts the weights of neurones in direction that minimises the gradient of a loss function [2]

Pros:

- Able to solve complex and non-linear problems.
- Works well with large data
- The same accuracy ratio can be achieved even with smaller data [3].

Cons:

- Computationally expensive to train the model
- The performance of the model is dependent on the quality of the training.
- Sensitive to feature scaling [3].

Support Vector Machines (SVM):

A Support Vector Machine is a non-parametric supervised learning model [4] which attempts to classify data points by finding the hyperplane of maximum margin by separating the different classes in the dataset. It is typically used in binary classification tasks, SVMs can be generalised to non-binary classification problem via 1 Vs 1 or 1 vs all classification algorithms [4].

Pros:

- Effective in high dimensional space
- Still effective in cases where number of dimensions is greater than the number of samples.
- Different kernel functions able to be used, making it versatile.

Cons:

- It is easier to overfit data
- SVMs are deterministic; meaning that they do not provide probability estimates for their predictions.

Hypothesis

From research paper [6] that I have read, we can see that the best performing model is the MLP classifier, with a test accuracy of 81.32%, whilst the SVM model gave an accuracy of 61%. Comparing the SVM results with paper [7], which looked at a variety of SVM models we can see that the highest test accuracy model was with kernel type RBF, which had results of 86.13%. From these

two papers, I expect both models to perform well and achieve an high testing accuracy for my dataset; however, SVM once the hyperparameters are optimized will outperform the MLP model.

Dataset

The dataset we have chosen, downloaded from Kaggle, looks at water quality from 3726 different samples, to decide whether the water is safe to drink or not. The decision whether the water source is safe to drink or not is based on 9 features (pH Value, Hardness, Solids, Amount of Chlorine, Sulphate, Conductivity, organic carbon, Trihalomethanes and turbidity) and has 2 possible output classes (1- Safe to drink the water, 0-Not safe to drink the water).

Table 1: showing summary statistics of the features of the dataset.

(0) - 1200					(1) - 811				
Features	Mean	s.d	Minimum	Maximum	Features	Mean	s.d	Minimum	Maximum
Ph Value	7.07	1.66	1.43	14.00	Ph Value	7.17	1.44	0.23	11.90
Hardness	196.01	30.72	98.45	300.29	Hardness	195.91	35.30	73.49	317.34
Solids	21,628.54	8,461.11	320.94	55,334.70	Solids	22,344.92	8,891.55	1,198.94	56,488.67
Amount of Cl	7.11	1.48	2.46	12.65	Amount of Cl	7.17	1.73	1.39	13.13
Sulfate	333.74	36.40	203.44	460.11	Sulfate	332.46	47.45	129.00	481.03
Conductivity	427.55	79.88	210.32	753.34	Conductivity	425.01	81.95	201.62	695.37
Organic carbon	14.40	3.37	4.37	27.01	Organic carbon	14.29	3.26	2.20	23.60
Trihalomethanes	66.28	15.93	14.34	120.03	Trihalomethanes	66.58	16.30	8.58	124.00
Turbidity	3.96	0.78	1.45	6.49	Turbidity	3.99	0.78	1.49	6.49

The dataset has been pre-processed to ensure that instances which have missing data are removed from the dataset, which left us with 2011 instances. From this 1200 are of type not safe to drink and 811 are instances where water is safe to drink as shown in Table one. Looking at Figure 1, showing the correlation matrix of all the features; we can see that there is little correlation between the features with highest value being -0.17 between solids and sulphates.

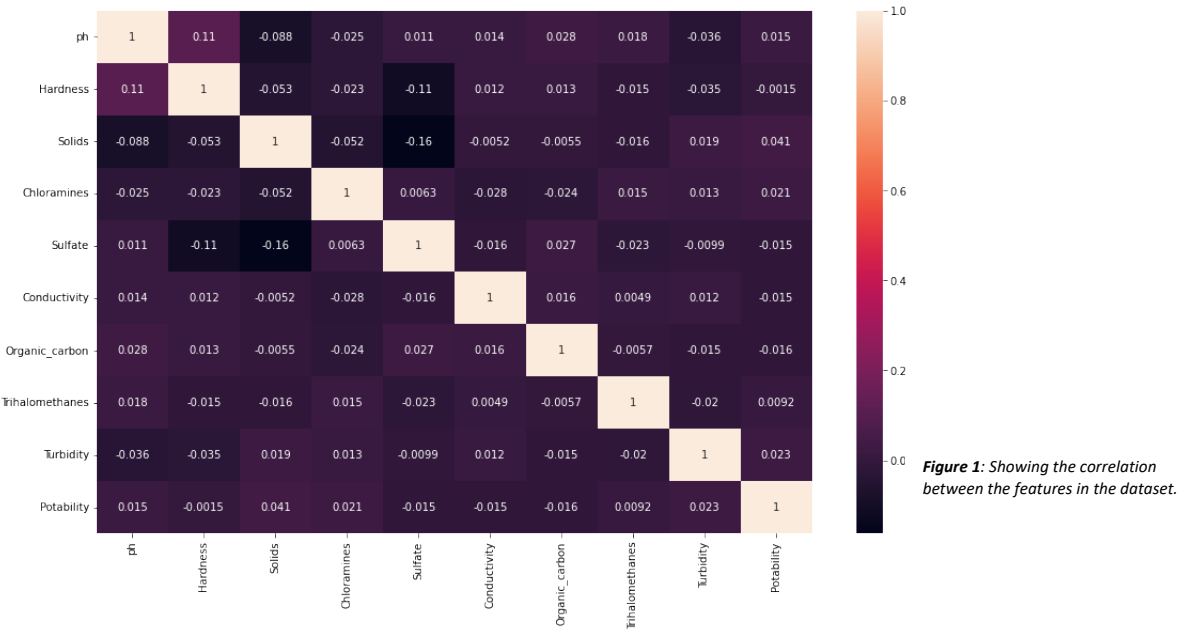


Figure 1: Showing the correlation between the features in the dataset.

Methods

This section will be looking at the approach that will be taken to train, validate and test both models, as well as providing a description of the hyperparameters that will be used for each respective model in search of the optimal configuration.

Methodology:

The dataset, after scaling, was split into training and testing into proportion of 80% and 20% respectively with a random state of 10. The testing data will be unseen until the best hyperparameters have been chosen for the respective model.

To ensure fair comparison grid search approach will be applied to both models to adjust and optimize the hyperparameters. As well as that, a K-fold cross validation will be applied to evaluate the performance of the models, as this will, be better for generalisation of the model performance against the unseen data. As I am predicting a close accuracy for both models, I will be using this search ahead of random search hence will be looking at all the possible outcomes based on the chosen hyperparameters. As this will be computationally expensive a K-fold of 3 will be used and the average of the K-folds will be used model accuracy.

To be able to compare performance of the chosen models using the test set, metrics such as accuracy, precision recall & F1 will be used. In addition to this confusion matrix will also be produced to further assess the performance of the misclassified data. The time taken to train the models will also be investigated.

Architecture and Parameters used for the MLP

Our model will consist of two hidden layers, fully connected, with a sigmoid activation function. The input layer and output layer will remain constant, while the hyperparameters such as learning rate, hidden layers, momentum, weight decay and dropout will be varied. It was decided to use a maximum of two layers only as having more than 3 layers is mainly for the complex representations [9]

For the loss criterion, a cross entropy loss function was chosen as it is more effective for binary classification classifiers. The weights will be updated using backpropagation using the 'stochastic gradient descent' (SGD) as the optimizer with momentum.

An early stopping criterion was used to prevent overfitting so that training stops if the validation set does not improve for a certain number of epochs, this will help prevent overfitting.

A grid search will be implemented. To give us an understanding of how the hyperparameters are affecting the model and then a grid search will be applied again with a change of hyperparameters based on the results of the first grid search.

Architecture and Parameters used for the SVM

The choice of hyperparameters for SVMs is significantly less when compared to MLP therefore, the following hyperparameters will be tuned. The type of kernel (linear, rbf & polynomial), box constraint C and gamma.

A trial-and-error approach will be initially implemented to find the optimum kernel, and from the different values of gamma and kernel scale will be used. Once an optimum value is found a grid search will be used for a range of hyperparameters based on the previous results to find the optimum generalisation model.

Results, Findings & Evaluation

Validation:

SVM:

From the as mentioned process rbf was the kernel which gave the best accuracy. The hyperparameters following the grid search two models gave the best accuracy from the grid search ($C=700$, $\gamma=0.01$) and ($C=1000$, $\gamma=0.01$) however the model with $C=700$ took much quicker to perform. Furthermore, looking at the results, the accuracy of the top 10 performing models varied by 0.94% hence performed with an similar accuracy and differed by a small amount. From the results, it can also be seen that as the kernel increased; the longer it took to train the model, hence becoming less efficient.

MLP:

From the first grid search, the top 5 performing models in terms of validation accuracies varied by only 0.62%, with the highest being an accuracy of 60.51%. The hypermeters for this model are (learning rate:1, dropout=0, hidden layer 1 = 4, hidden layer 2 = 4, momentum = 0.9 and weight decay = 0). As well as having the highest accuracy the model also took the quickest time to train. From the 3rd best performing model to the 10th best had the same validation accuracy.

For the 2nd gris search, weight decay hyperparameter was dropped as it had little effect on the best performing models and the dimensionality of the other hypermeters was increased. The best performing model for this validation accuracy,60.01%, was slightly lower than the previous grid search however what has significantly improved was the time taken for the models to train. As the model from grid search 2 was more efficient the hyperparameters will be put forward for the unseen data.

Training & Testing:

From figure two we can infer that both models had similar accuracies to their training data and the testing time when compared to the training was also quicker. SVM performed quicker in training however in testing it took much longer when being compared with the MLP.

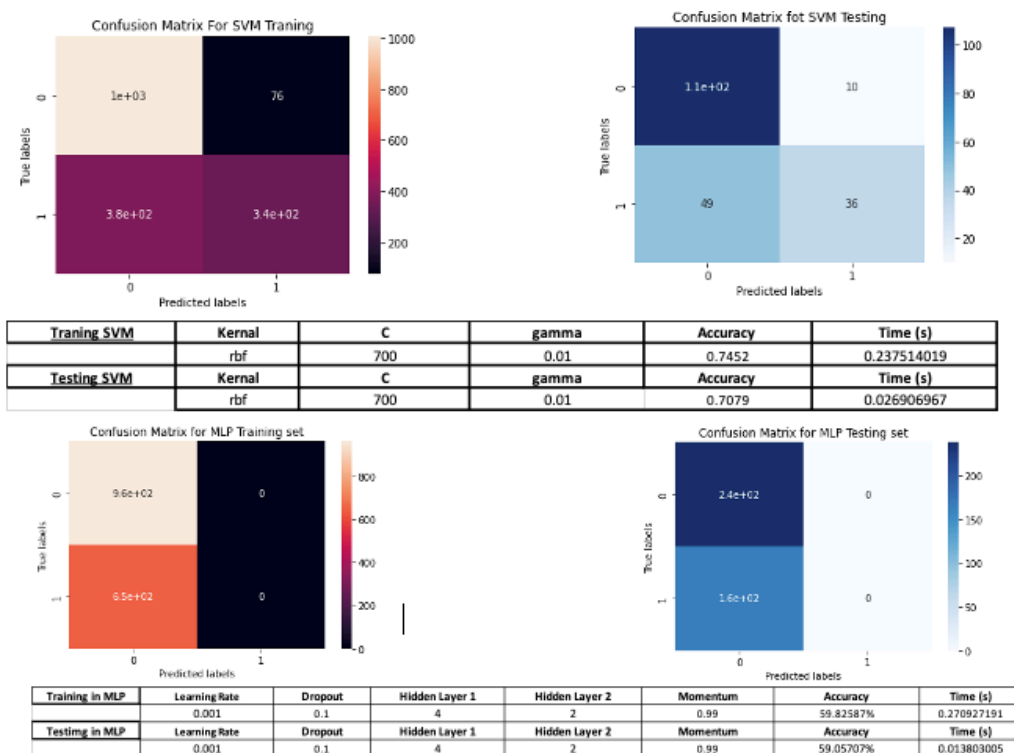


Figure 2: Showing the confusion matrix and results for the chosen hype parameters for SVM and MLP.

Looking into the confusion matrix in figure 2, SVM misclassified data as much as 5 times more for the false negative than the false positive for both the training and testing whereas, for MLP it was unable

to predict anything related to the actual positive and actual negative classes, hence is very imbalanced.

Further analysing the performance metrics for the test data, from classification report, SVM has a higher precision for both classes, hence is better at predicting true positives amongst all the samples predicted positive however this was expected considering the MLP model did not predict a single outcome for class 1. MLP did however outperform the SVM in the recall for class 0.

From figure 3 and 4, we see the ROC curve for both our models which further illustrates the point made regarding the performance of the MLP model, with an AUC value of 0.53 which tells us that the model is only slightly better than if you were to random guess the output.

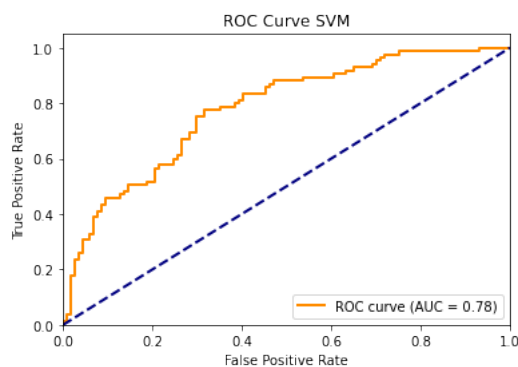


Figure 3: Showing the ROC curve for the SVM model against the testing data.

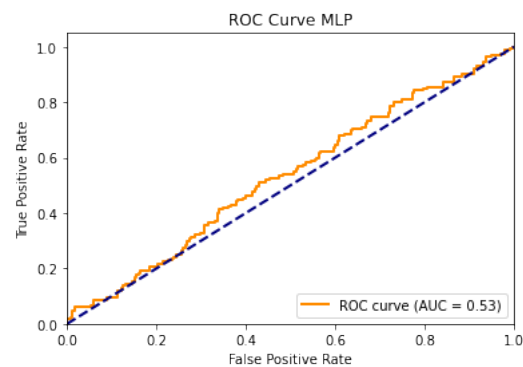


Figure 4: Showing the ROC curve for the MLP model against the testing data.

Conclusion, Lessons learnt & Future Work

In conclusion we learnt that the SVM was better at predicting whether the quality of water was safe to drink or not, however the SVM was not perfect with it showing some biasness towards the false negative values. Comparing my results to the papers I have read, my results were less accurate, however, this may be due to the dataset not being entirely the same.

What I have learnt that having more hyperparameters in a model does not mean that it would perform any better, having only tuned 3 hyperparameters in my SVM model, with 6 being used for the MLP model. As well as that checking the quality of the results from the training stage will ensure that the best performing model is put forward, so just because a model is showing high accuracy, does not mean it will generalise well, even though K Fold is used.

Going forward I would like to use a computer with more GPUs as this will allow me to use a wider range of hyperparameters and a K 10/15 K-fold Cross validation. Also get more data so that training size is able to increase. In addition to this, able to combine both models to create a hybrid [12].

Intermediate Results

SVM:

1. Choosing Kernal type stage against the default parameters:

Kernal	Accuracy
rbf	75.23
Linear	59.87
Poly	64.68

2. Random search of the hyperparameters – trial and error

Kernal	C	gamma	Accuracy
rbf	0.01	0.001	59.8700000%
rbf	0.01	0.01	59.8700000%
rbf	0.1	0.01	59.8700000%
rbf	0.1	0.1	60.5300000%
rbf	10	0.1	83.5800000%
rbf	100	0.1	94.2000000%
rbf	110	0.1	94.4200000%
rbf	120	0.1	94.4200000%
rbf	150	0.1	94.8000000%
rbf	170	0.1	95.0800000%
rbf	200	0.1	95.5800000%
rbf	300	0.1	96.7400000%
rbf	400	0.1	97.4600000%
rbf	500	0.1	97.7900000%
rbf	600	0.1	98.1200000%
rbf	750	0.1	98.5100000%
rbf	1000	0.1	99.0000000%
rbf	2000	0.1	99.5000000%
rbf	2000	1	100.0000000%

3. Grid search, showing the top 10 models inc time

hyperparameters =

{'C':[10,500,700,1000,1200,1300,1400,1500,1600,1700,1800,1900,2000,2500,2700,2900,3000], 'kernel':['rbf'], 'gamma':[0.01, 0.1,1]},

Rank	C	gamma	Accuracy	Time (s)
1	700	0.01	0.6705	0.657280207
2	1000	0.01	0.6705	0.938501358
3	1200	0.01	0.6683	0.97936511
4	1300	0.01	0.6656	1.09807682
5	500	0.01	0.665	0.484737396
6	1500	0.01	0.6639	1.401268721
7	10	0.1	0.6633	0.259310007
8	1400	0.01	0.6633	1.046247482
9	1600	0.01	0.6633	1.245649815
10	1700	0.01	0.6611	1.243689537

Test performance classification result SVM:

precision recall f1-score support

class 0 0.69 0.91 0.78 117
class 1 0.78 0.42 0.55 85

accuracy			0.71	202
macro avg	0.73	0.67	0.67	202
weighted avg	0.73	0.71	0.69	202

MLP:

The results from the 1st grid search

```
params = { 'lr': [0.001, 0.1], 'module__hidden_size1':[2,4,6], 'module__hidden_size2':
[0,2,4,6], 'optimizer__weight_decay': [0,0.001, 0.1], 'optimizer__momentum': [0,0.3,0.9],
'module__dropout_rate': [0,0.3,0.8], }
```

Rank	Learning Rate	Dropout	Hidden Layer 1	Hidden Layer 2	Momentum	Weight Decay	Accuracy	Time (s)
1	0.1	0	4	4	0.9	0	60.50995%	0.800211191
2	0.001	0	2	4	0.3	0	60.26119%	2.066486756
3	0.001	0.8	6	6	0.3	0	60.19900%	2.285940011
4	0.001	0.3	6	2	0.3	0	60.01244%	2.153249502
5=	0.001	0.3	2	6	0	0.1	59.88806%	1.620339235
5=	0.001	0.8	6	6	0.3	0	59.88806%	1.769641638

2nd Grid search results

Rank	Learning Rate	Dropout	Hidden Layer 1	Hidden Layer 2	Momentum	Accuracy	Time
1	0.001	0.1	4	2	0.99	60.01244%	0.267951171
2	0.001	0.1	4	4	0.99	59.88806%	0.267951171
3	0.001	0.001	2	2	0.99	59.82587%	0.257312536
3	0.001	0.001	2	3	0.1	59.82587%	1.489685376
3	0.1	0.001	2	3	0.2	59.82587%	1.983034849
3	0.1	0.001	2	3	0.99	59.82587%	0.444810629

```
params = {
'lr': [0.001, 0.1], 'module__hidden_size1':[1,2,4,6], 'module__hidden_size2': [1,2,3,4],
'optimizer__momentum': [0.1,0.2,0.6,0.9,0.99], 'module__dropout_rate': [0.8,0.9],
}
```

Test performance classification report:

	precision	recall	f1-score	support
class 0	0.59	1.00	0.74	238
class 1	0.00	0.00	0.00	165
accuracy			0.59	403
macro avg	0.30	0.50	0.37	403
weighted avg	0.35	0.59	0.44	403

Glossary:

Dropout: is an regularisation technique which is used to prevent overfitting by randomly dropping selected neurones during training, which prevents the neurones from co-adapting too much [10]

Weight Decay: A regularisation technique to prevent the network from overfitting the data. works by adding a penalty term to the cost function of a neural network which has the effect of shrinking the weights during backpropagation [11].

Hyperparameter: A type of parameter that you are able to change before the training process begins and used to control learning process; e.g. learning rate.

Generalisation: Is the ability of the model to adapt to data which it has not been trained on.

Accuracy: It is the proportion of correctly predicted observations to all observations. True negative/ true positive/ Total

Fully connected Network: A Fully connected network is where all the nodes/neurons in one layer are all connected to the neurons in the next layer

Recall: Relatively how many correctly predicted positives there are compared to all. True positive/true positive + false negative

Precision: It is the proportion of correctly predicted positive observations to the total number of expected positive observations. True positive/ true positive plus false positive

F1 – Score: It is the weighted average of precision and recall. Defined by $2 * (\text{Recall} \times \text{Precision}) / (\text{Recall} + \text{Precision})$

References

[1] <https://iwaponline.com/wqrj/article/57/3/152/88998/Machine-learning-for-water-quality-classification> (Introduction)

[2] <https://www.educative.io/answers/what-is-a-multi-layered-perceptron> [MLP]

[3] Comparison of Multi-class Classification Algorithms on Early Diagnosis of Heart Diseases - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/Multilayer-Perceptron-Advantages-and-Disadvantages_tbl4_338950098 [MLP]

[4] <https://paperswithcode.com/method/svm>

[5] <https://scikit-learn.org/stable/modules/svm.html> [SVM]

[6] Nair, Jitha P., and M. S. Vijaya. 'River Water Quality Prediction and Index Classification Using Machine Learning'. *Journal of Physics: Conference Series* 2325, no. 1 (August 2022): 012011. <https://doi.org/10.1088/1742-6596/2325/1/012011>. [hyp]

[7] Liao, Yue, Jianyu Xu, and Wenjing Wang. 'A Method of Water Quality Assessment Based on Biomonitoring and Multiclass Support Vector Machine'. *Procedia Environmental Sciences* 10 (2011): 451–57. <https://doi.org/10.1016/j.proenv.2011.09.074>. [hyp]

[8] Setshedi, K.J.; Mutingwende, N.; Ngqwala, N.P. The Use of Artificial Neural Networks to Predict the Physicochemical Characteristics of Water Quality in Three District Municipalities, Eastern Cape Province, South Africa. *Int. J. Environ. Res. Public Health* **2021**, *18*, 5248. <https://doi.org/10.3390/ijerph18105248> [c1]

[9] S. Yadav and S. Shukla, "Analysis of k-Fold Cross-Validation over Hold-Out Validation on Colossal Datasets for Quality Classification," *2016 IEEE 6th International Conference on Advanced Computing (IACC)*, Bhimavaram, India, 2016, pp. 78-83, doi: 10.1109/IACC.2016.25.URL: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7544814&isnumber=7544788> [K fold]

[10] Srivastava, Nitish & Hinton, Geoffrey & Krizhevsky, Alex & Sutskever, Ilya & Salakhutdinov, Ruslan. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*. 15. 1929-1958. https://www.researchgate.net/publication/286794765_Dropout_A_Simple_Way_to_Prevent_Neural_Networks_from_Overfitting/citation/download

[11] <https://programmatically.com/weight-decay-in-neural-networks/> [weigh decay]

[12] title={A Hybrid MLP-SVM Model for Classification using Spatial-Spectral Features on Hyper-Spectral Images},author={Ginni Garg and Dheeraj Kumar and ArvinderPal and Yash Sonker and Ritu Garg},
<https://arxiv.org/abs/2101.00214>